



PAK-AUSTRIA FACHHOCHSCHULE

INSTITUTE OF APPLIED SCIENCES AND TECHNOLOGY



Name : Hafsa Shahid

Reg no : B25F34CYS101

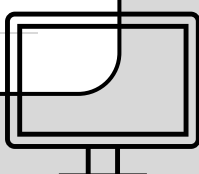
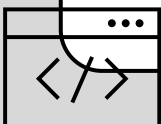
Program : Cybersecurity

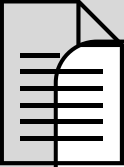
Instructor's name: Hassan Ali

Date Submitted : 17 / 07 / 26

Deadline : 17 / 07 / 26

Assignment Report # 7





• INTRODUCTION •

In this assignment, we explored the concepts of Web Application Security through a few tasks. The objective was to understand how web applications function, and how common security vulnerabilities can be identified and exploited. The assignment consists of three tasks; it goes as follows

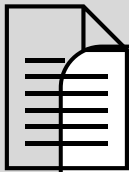
Assignments Tasks:

Task #01 : *Web Application Reconnaissance & Technology Identification*, choose an authorized test website Identify:

- Website URL
- HTTP vs HTTPS
- Request methods used (GET, POST)
- Response status codes (200, 301, 302, 403, 404, 500)
- Technologies used (server, CMS, framework) using tools like WhatWeb or Wappalyzer.
- Capture screenshots of your findings.
- Write a short report (1–2 pages) explaining your observations.

Task #02 : *HTTP Requests, Authentication & Authorization Analysis*, Open an authorized web application. Use your browser's Developer Tools to inspect requests. Identify:

- HTTP request headers
- HTTP response headers
- Cookies
- Session ID
- Request methods (GET, POST, PUT, DELETE if present)
- Explain the difference between: **Authentication** and **Authorization**
- Tokens (JWT)
- Draw a simple authentication flow diagram.



Task #03 : *SQL Injection*, Use an intentionally vulnerable application such as <http://65.61.137.117/>. Review the SQL Injection concept. Perform basic SQL Injection testing on the provided login or search page. Record:

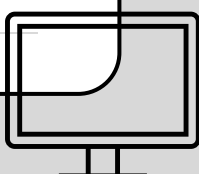
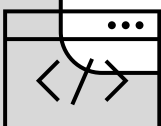
- The vulnerable input field
- Payload(s) used
- Application response
- Why the vulnerability exists
- How parameterized queries (prepared statements) prevent SQL Injection
- Provide screenshots of each step.

• TASK # 01 •

Task #01: *Web Application Reconnaissance & Technology Identification*, choose an authorized test website.

Identify:

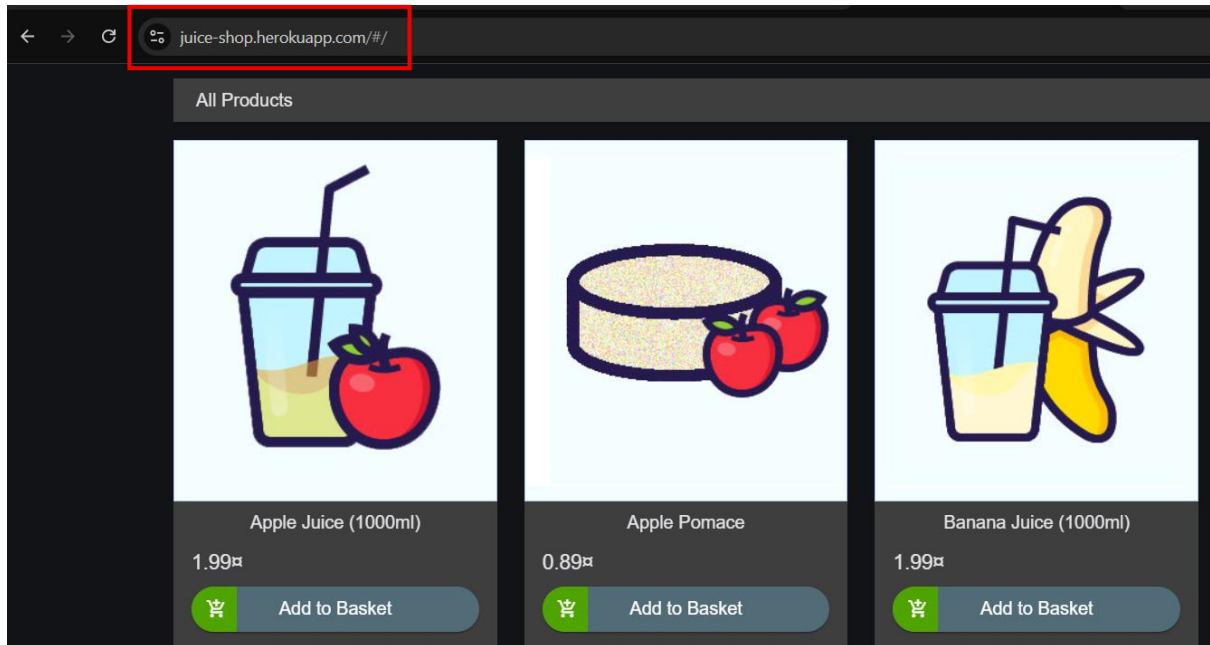
- Website URL
- HTTP vs HTTPS
- Request methods used (GET, POST)
- Response status codes (200, 301, 302, 403, 404, 500)
- Technologies used (server, CMS, framework) using tools like WhatWeb or Wappalyzer.
- Capture screenshots of your findings.
- Write a short report (1–2 pages) explaining your observations.



Website URL :

The URL is found at the top for the OWASP Juice Shop the URL is,

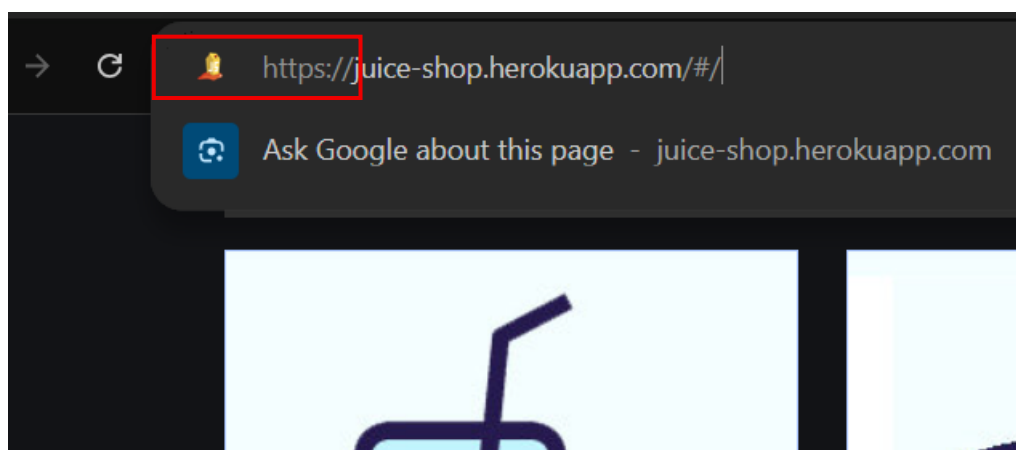
<https://juice-shop.herokuapp.com/#/>



Task # 01: URL of Website

HTTP vs HTTPS :

The URL found at the top for the OWASP Juice Shop includes **HTTPS** instead of **HTTP**. The major difference between these is that in HTTPS, the browser and server first establish a secure connection then exchange data.

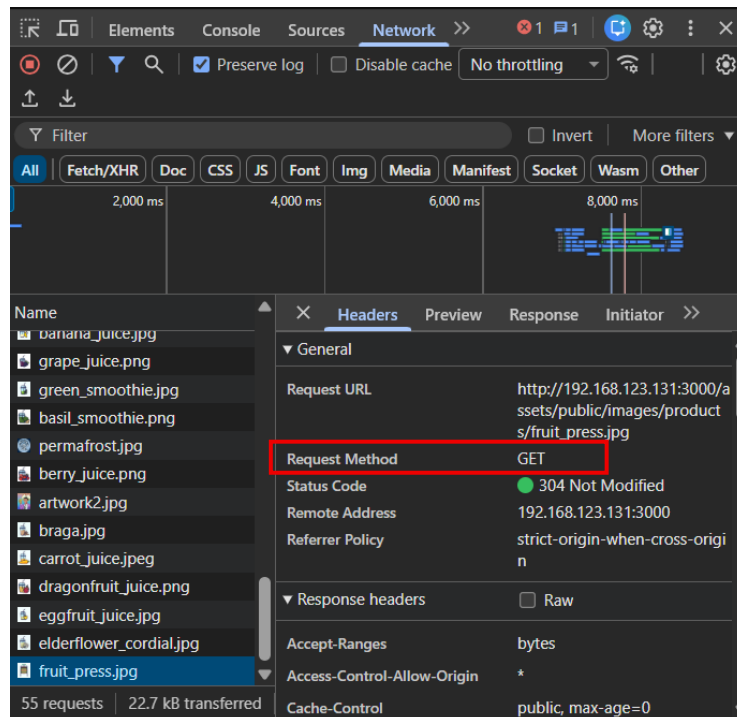


Task # 01: HTTPS used in URL of Website

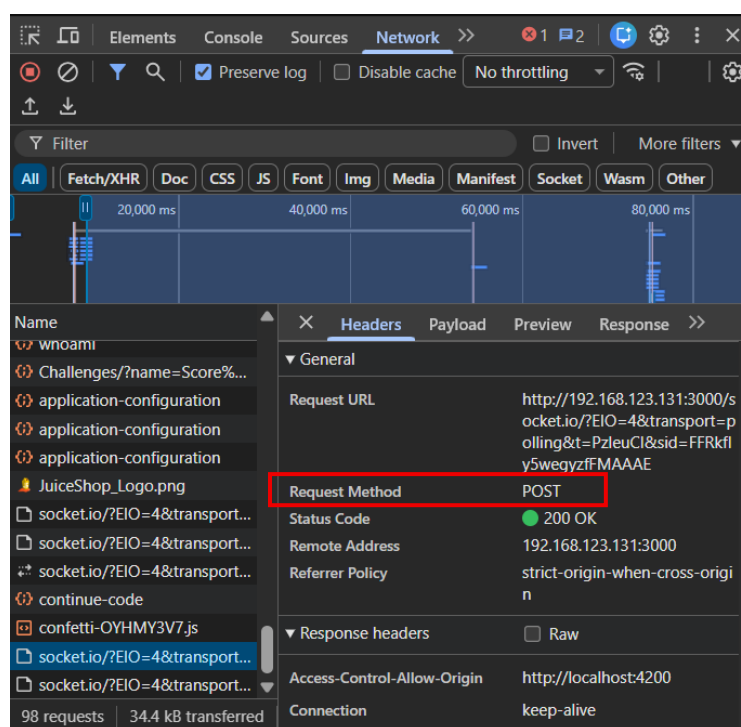
Request Method :

Request Methods are HTTP commands that tell the server what action to perform. Using Developer Tool. The following request methods have been observed,

- **GET:** Its purpose retrieve information from server. EXAMPLE, getting an image.
- **POST:** Its purpose is to send data to server. EXAMPLE, login info.



Task # 01: Request Method GET used in Website

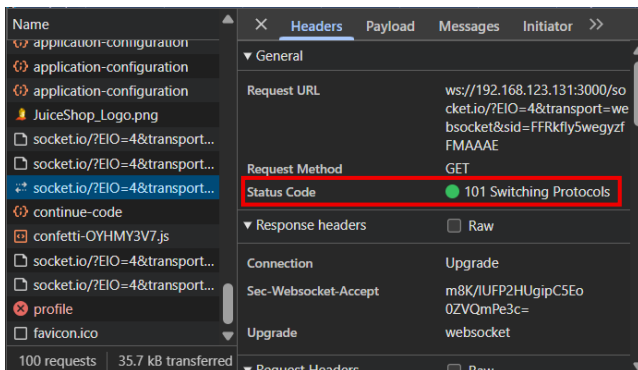


Task # 01: Request Method POST used in Website

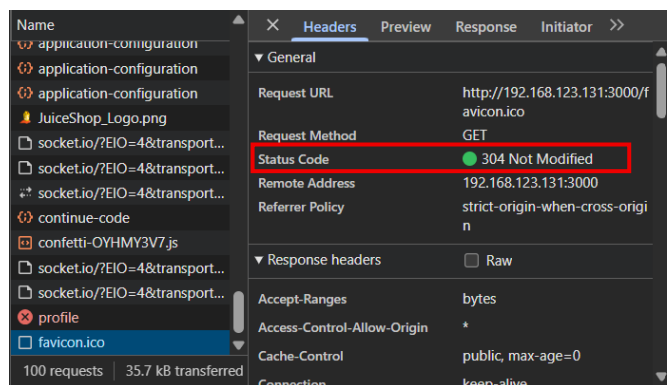
Response Status Codes :

HTTP Response Status Codes are three-digit numbers returned by the server to indicate the result of a client's request. The following response status codes have been observed,

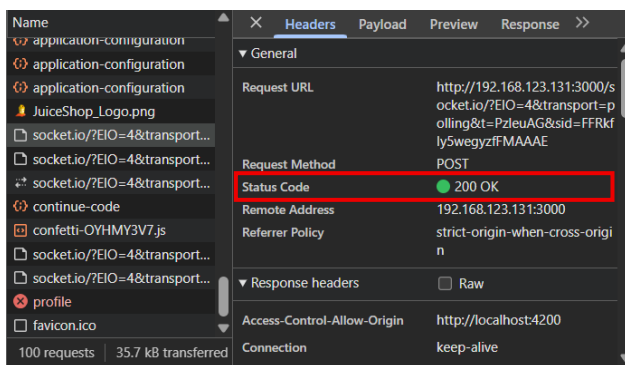
- **200** : Request successful
- **101** : Switching Protocols.
- **304** : Not Modified.
- **500** : Server Error.



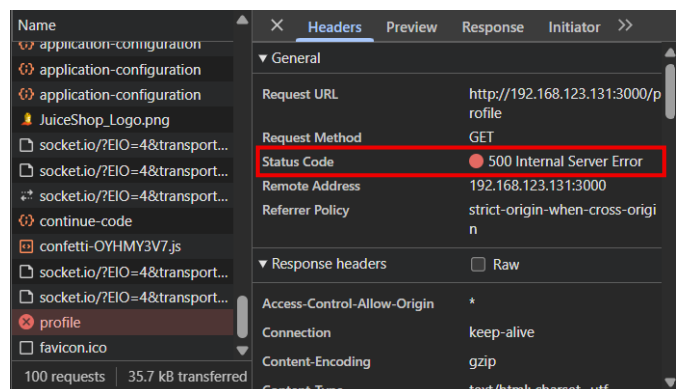
Task # 01: Response Code 101



Task # 01: Response Code 304



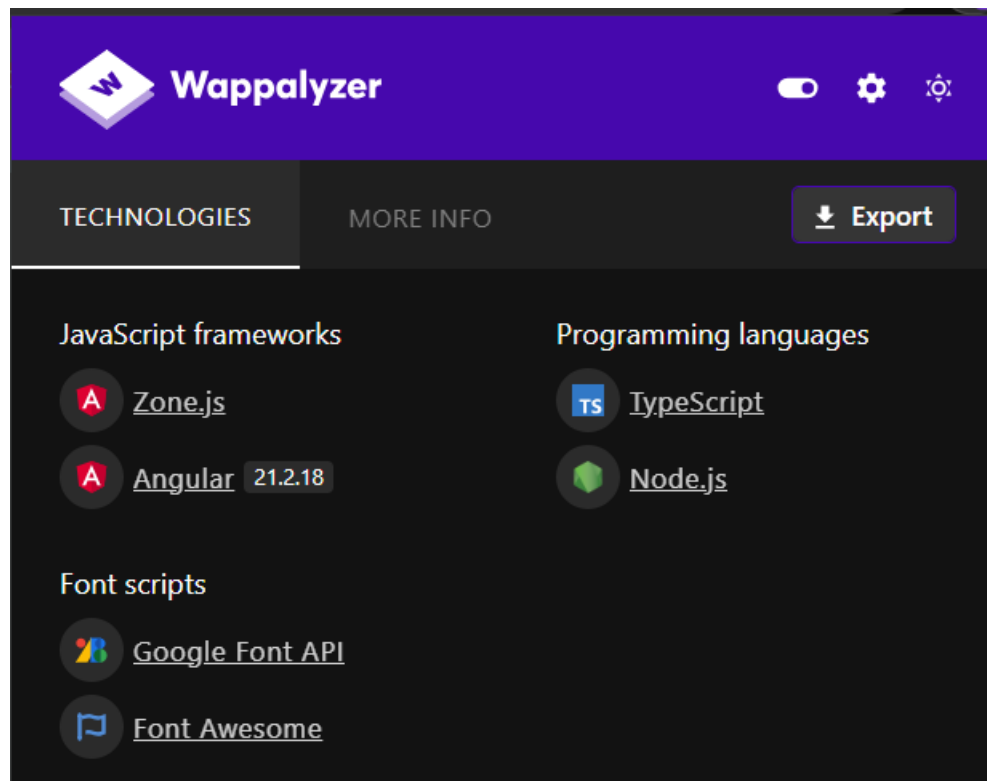
Task # 01: Response Code 200



Task # 01: Response Code 500

Technologies used :

I used Developer Tools and the browser extension Wappalyzer to inspect the website. The website itself uses these tools,



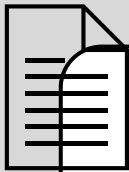
Task # 01: Tools used in Website

Task Conclusion :

In this Task we successfully looked the background of a website Juice Shop. We looked it its,

- URL.
- Request Methods.
- Response Methods.
- The tools the Website uses

We learned how to use developer tools.



• TASK # 02 •

Task #02: *HTTP Requests, Authentication & Authorization Analysis*, Open an authorized web application. Use your browser's Developer Tools to inspect requests. Identify:

Identify:

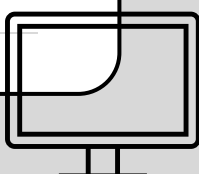
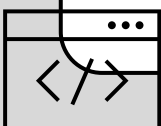
- HTTP request headers
- HTTP response headers
- Cookies
- Session ID
- Request methods (GET, POST, PUT, DELETE if present)
- Explain the difference between: **Authentication** and **Authorization**
- Tokens (JWT)
- Draw a simple authentication flow diagram.

I am going to inspect the <https://www.reddit.com/> Website.

HTTP Request Headers :

HTTP Request Headers are additional information sent by the browser to the server along with each request, such as browser type, cookies, accepted content types, and authorization details. Following is some of the headers I found,

- **Method:** Get.
- **path:** /echo.
- **Scheme :** https.
- **Cookie :** a long string...
- **Authority :** reqbin.com.
- **Cache-Control :** max-age=0.
- **Priority :** u=0,i





Name	Value	Domain	Path	Expire...	Size	HttpO...	Secure	Same...	Partiti...	Cross ...	Priority
__Host-1PLSID	o.chromewebstore.google.com o.drive.googl...	accou...	/	2027-...	264	✓	✓				High
__Host-3PLSID	o.chromewebstore.google.com o.drive.googl...	accou...	/	2027-...	264	✓	✓	None			High
__Host-GAPS	1:NU8_jPgVu3jFmGuZ7dl3jvnfV05GaXdaa1h...	accou...	/	2027-...	124	✓	✓				High
__Secure-1PAPISID	eUTGf9N-0V5zExTB/AaVCOqfgDMdk8bBkC	.googl...	/	2027-...	51		✓				High
__Secure-1PSID	g.a000_wgQf7v13pgDC9MznnAWPGHYNUg...	.googl...	/	2027-...	167	✓	✓				High
__Secure-1PSIDCC	AKEyXzUM60bYI9_qLwxxwIbOu8BWNcy_eos...	.googl...	/	2027-...	90	✓	✓				High
__Secure-1PSIDRTS	sids-CjYBPWEu2auaA1MOeHisIDP-iHTDOuu...	.googl...	/	2026-...	101	✓	✓				High
__Secure-1PSIDTS	sids-CjYBPWEu2auaA1MOeHisIDP-iHTDOuu...	.googl...	/	2027-...	100	✓	✓				High
__Secure-3PAPISID	eUTGf9N-0V5zExTB/AaVCOqfgDMdk8bBkC	.googl...	/	2027-...	51		✓	None			High
__Secure-3PSID	g.a000_wgQf7v13pgDC9MznnAWPGHYNUg...	.googl...	/	2027-...	167	✓	✓	None			High
__Secure-3PSIDCC	AKEyXzXZEmErmbpwZmZherdm2Vn01UR-zz...	.googl...	/	2027-...	91	✓	✓	None			High
__Secure-3PSIDRTS	sids-CjYBPWEu2auaA1MOeHisIDP-iHTDOuu...	.googl...	/	2026-...	101	✓	✓	None			High
__Secure-3PSIDTS	sids-CjYBPWEu2auaA1MOeHisIDP-iHTDOuu...	.googl...	/	2027-...	100	✓	✓	None			High
__Secure-BUCKET	CFY	.googl...	/	2027-...	18	✓	✓				Mediu.
ACCOUNT_CHOOSER	AFx_qI5-MglGmjicLP7jxc0q6EON1bmokE...	accou...	/	2026-...	155	✓	✓				High
ads_cookie	1	www.r...	/	2027-...	11			Strict			Mediu.
AEC	AdJVEaupSI0ZBcH3c9Vvr8GVGfgJ3yYYUqfSZ...	.googl...	/	2026-...	61	✓	✓	Lax			Mediu.
APISID	d1EoQ5k8L6qKR_V/ANqjMhDK6wNthK0D-	.googl...	/	2027-...	40						High
csrf_token	a9c12af92c180f8e1b2d25022b1cf331	.reddit...	/	Session	42		✓	Strict			Mediu.
csv	2	.reddit...	/	2027-...	4		✓	None			Mediu.
DV	I5mrUSi-rsFhACJ-bZgLX55dLsgI95Iw4P5MyR...	www....	/	2026-...	112						Mediu.
edgebucket	WN3CVgHLPmjJ3wmI5U	.reddit...	/	2027-...	28		✓				Mediu.
g_state	("LJ":0,"LI":1784299033469,"I_e":["enable_itp...	www.r...	/	2027-...	147						High
HSID	AGL-ZsKdSm7vaBO0u	.googl...	/	2027-...	21	✓					High
Ioid	0000000024rex2k75b.2.1766659393490.Z0FB...	.reddit...	/	2027-...	226		✓	None			Mediu.
LSID	o.chromewebstore.google.com o.drive.googl...	accou...	/	2027-...	255	✓	✓				High
LSOLH	_SVI_EMKKhPiokZUDGAMiP01BRURIZi9xyJd0...	accou...	/	2027-...	127		✓	None			Mediu.
NID	533=hjYA2hxzdG5sEDohVw38xB_JwKa_E9-B...	.googl...	/	2027-...	373	✓	✓	None			Mediu.
OTZ	8658332_36_36_36_	accou...	/	2026-...	21		✓				Mediu.
pxrc	CMX76NIGegYI3N0qEFM=	.rldn...	/	2026-...	24		✓	None			Mediu.
reddit_supported_media_codecs	video/avc2Cvideo/vp9	www.r...	/	2027-...	50						Mediu.
rlas3	eOKI8gcpjikTtS85y6vZqUDbFw5dJepetJhoy...	.rldn...	/	2027-...	49		✓	None			Mediu.
SAPISID	eUTGf9N-0V5zExTB/AaVCOqfgDMdk8bBkC	.googl...	/	2027-...	41		✓				High

A Session ID is a unique identifier generated by the server and stored in a cookie to track and maintain a user's active session across multiple requests. Following is the Session ID highlighted,

```
Cookie
loid=0000000024rex2k75b.2.1766659393490.Z0F8QUFBqnBUUIZCZ3Jfclck1SXdenFPQjOulFkcbz4bKqYbV4VIRWbHFMuUpdEdVaVj
Qz2zdxJRtMj2ThtkXmBwZ0FNhCvLdV9leGp5sAlFRtZcZUZXV1I2Z0k3SEZLjMjSWC3v28T3NRbkVz5khlMnVSTENjYk10Sn8sW
Gx3ZzdCnRm: csv-2: edgebucket=WN3CvqHLPmj3wml5U: reddit_supported_media_codes=video/avc%2Cvideo/vp9;
token_v2=eYlhbGGoUj5Uu21NilsimtpZC6lilNIQT11NjpxS3dsMnlisV0VtMjVmcXhwTU40cWY4MXE2OWFFdWfWfYmnpLMUdhVgjdWZNziw
idHwVjioiUdln0.eYjzdvWlI05u2DKliwZvHljoxNzg0Mzg1MzlyLjg50TQwNcWiaWF0joxNzg0MjktOTYjLjg50TQwMywianRpjoiZc3BUU
tYk9vU0w0b1S09GvW53Y0R0LVZqAV8VlIuY2kIjoiMFltV0FNhVvby1NeElCjsaWQjO0U0Ml8NHJleDJRnZlilwiwbGNhJioxNzY2NjU
5MzkuZndKwLClj3AIOIUSnhraZHT3REQU0Lk3T1GYTVZ2Y1Vv9MDF0Y1lnc0XRy9NwM243RFZv23YmDjDRJwSFASREtvcUZEQj1pYz
3FQUWjGj1rVERCUUvUW05MbtNmMlOZThXpTh0XpN3RkRya21R3pmpUvFz2UjYXk4215VdXUe234XRHTksMMDVBR8Nf
oY0lYzUZlZicGMybZrYmk1NmRlRk1ckR5b3NWZmwgdGPHRlksZbnhgY2hdxdJwU2mk1rbksdRktdWHZUak45VzM5dm161X1NhMe
4D0txdW1CM2hsKlNHNHnMcGltM2Q5V5y3N1NrDeGexOTNxtUTJ1ZDYZsU5MWM3M83ZWY2k2xy5XhWfJpA1KdnQcMXktaEEODh
MelBxQUVhczRVY1pkbVFkQ2xVSFVMbWdKR01KNHRNSTVMcmwyMzhKdG61ZVHY4YnRfEjkt4TS1LbU5fDedT06Q2VMUXBFSDFHd0FBX
184UEtVFiFlCmbG8hOjF9.c8bV3YQj.Et4MU00rKjnGNaMiepoOpffd82ME_L09npYtClUqVvAie7H8G31N
CroulEAVXpYq9VE8iHlUffux1MqR7gyMpCc3jnkodzu0Qbwz0V2QYR1mpPmpQoTL6kBoQignKQ1OcltnF2W0alQlwwVfrMpCmP8G
nhlFzicv4D1pws_AmCYxzaL8nNfEouA1JgZJUE0Jkzvp1WIr90J8LUXqj7Ymyexifit24XSCTkS5AsuNi82zcRMnSzpy-
wZUXRNI95mzl5S83ABEPNArOjwM5C4eCa-n2qcXM6lrmezs9P1a_J3d4zoNrnJmNqngTh2KCa: ads_cookie=1;
csrf_token=a9c12af92c180f8eb2d2502b2cf331; g_state=["l":0,"lji":1784299033469,"le":
"fenable_itp_optimization":24,"l_et":1784299033469,"l_b":"","fdRc7TsU0U0Hw/CqvjsWfUAlUHX8XQa+xsG0oAiQa"];
session_tracker=ocdnkhqmeoagodagna.0.1784299086564.Z0F8QUFBqnBUjXajPjJlGcF93VQVB8WkVznhtTWQY2k0ztY5RDlnWd
B4cEjNN2tQbFvY21Vn52w4ekYNIYIzYhHQJTZCekhtZmwyUHIDY01GU0FjeVJWwmp5dW5sb3U1Um1RUU9KdchQWDDlVaj3yC23VWVG
OTh8ZU15lImxvEm5Ven2vYXl0R7L
```

Request Method :

Request Methods are HTTP commands that tell the server what action to perform. Using Developer Tool, Following Request Methods have been observed,

- **GET:** Its purpose retrieve information from server. EXAMPLE, getting an image.
- **POST:** Its purpose is to send data to server. EXAMPLE, login info.
- **OPTIONS :** Requests the permitted communication options.

▼ General	
Request URL	https://www.reddit.com/svc/shreddit/update-recaptcha?k=cG9wdWxhcncmb3JlZ3JvdW5kaW5nX2FjdGlvbWwyYUQ4MWEyYy1jMGMwLTRkZmYtYjY4Yy03NWQyMzVjMzNiYWU
Request Method	GET
Status Code	● 200 OK
Remote Address	151.101.141.140:443
Referrer Policy	strict-origin-when-cross-origin

Task # 02: Request Method GET

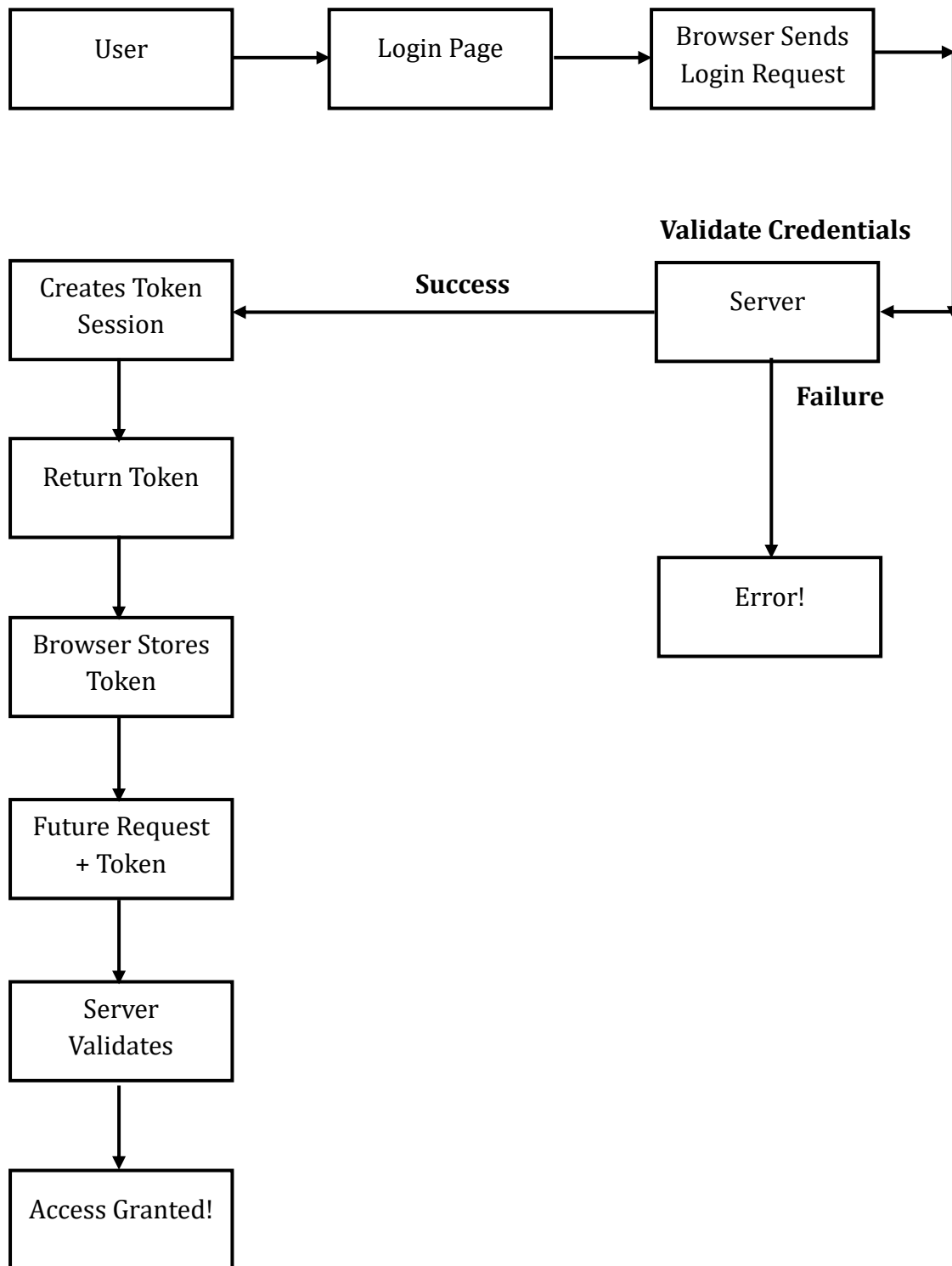
▼ General	
Request URL	https://www.reddit.com/svc/shreddit/events
Request Method	POST
Status Code	● 200 OK
Remote Address	151.101.141.140:443
Referrer Policy	strict-origin-when-cross-origin

Task # 02: Request Method POST

▼ General	
Request URL	https://pi.reddit.com/v4/p
Request Method	OPTIONS
Status Code	● 204 No Content
Remote Address	104.18.13.208:443
Referrer Policy	strict-origin-when-cross-origin

Task # 02: Request Method OPTIONS

Authentication Process Flow Diagram



Task Conclusion :

In this Task we analysed the requests of a website using devtools and looked at the difference between authentication and authorization.

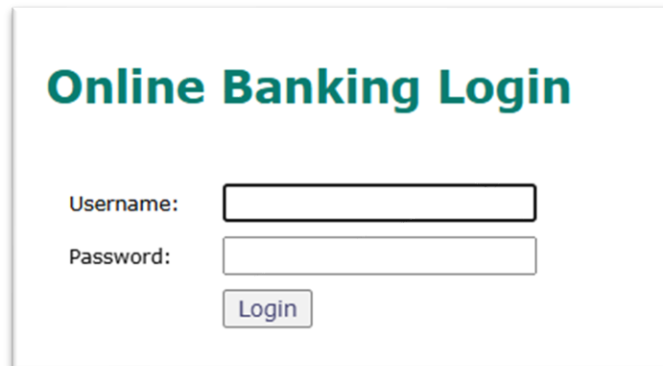
• TASK # 03 •

Task #03 : SQL Injection, Use an intentionally vulnerable application such as <http://65.61.137.117/>. Review the SQL Injection concept. Perform basic SQL Injection testing on the provided login or search page. Record:

- The vulnerable input field
- Payload(s) used
- Application response
- Why the vulnerability exists
- How parameterized queries (prepared statements) prevent SQL Injection
- Provide screenshots of each step.

Vulnerable Input Field :

The field on the webpage, which is susceptible to SQL Injection due to lack of input sanitization, Like a search bar or the username section of a login page.

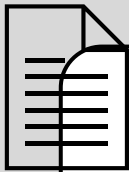


Task # 03: Login Page of Alturo Mutual

Payloads Used :

The Payload is a code that is entered in the Vulnerable Input Field with a value that bypasses the password field. I used the Payloads,

- ' OR 1=1--



Online Banking Login

Syntax error: Encountered "or1" at line 1, column 46.

Username:

Password:

Task # 03: Login Page of Alturo Mutual

Hello Admin User

Welcome to Alturo Mutual Online.

View Account Details:

Congratulations!

You have been pre-approved for an Alturo Gold Visa with a credit limit of \$10000!

Click [Here](#) to apply.

Task # 03: After Login Page of Alturo Mutual

- admin'--

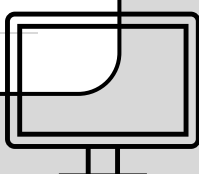
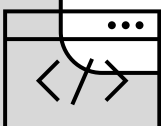
Online Banking Login

Syntax error: Encountered "or1" at line 1, column 46.

Username:

Password:

Task # 03: Login Page of Alturo Mutual



Hello Admin User

Welcome to Altoro Mutual Online.

View Account Details:

800000 Corporate



GO

Congratulations!

You have been pre-approved for an Altoro Gold Visa with a credit limit of \$10000!

Click [Here](#) to apply.

Task # 03: After Login Page of Altoro Mutual

Application Response :

When the payload (' OR 1=1--) or (admin'--) was entered in the username field and any value was entered in the password field, the login was successful. The application did not validate the credentials and granted access to the after screen. In both cases, I was logged in as the administrator user. After successful bypass, I could view account information and other privileged sections of the application.

Why the Vulnerability Exists :

The vulnerability exists because the website takes whatever you type in the username and **directly glues it** into the SQL command without checking it first. Instead of treating your input as normal text, the website treats it as part of the actual programming command. So when you type special characters, the website's command gets broken and changed, allowing you to trick the system into logging you in.

As a result, the application authenticates the user without verifying correct credentials.

Online Banking Login

Syntax error: Encountered "or1" at line 1, column 46.

Username: ' OR 1=1--
Password: ***

Hello Admin User

Welcome to Altoro Mutual Online.

View Account Details:

800000 Corporate



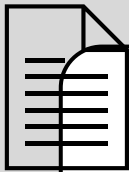
GO

Congratulations!

You have been pre-approved for an Altoro Gold Visa with a credit limit of \$10000!

[Click Here](#) to apply.

Task # 03: Before and After Login Page of Altoro Mutual



Parameterized Queries Prevent SQL Injection :

Parameterized queries solve this problem by separating the SQL code from the user data. Instead of concatenating strings, the query is prepared with placeholders, and the user input is sent separately as parameters. This works because the database engine treats the entire user input as a literal value, not as part of the SQL code, making the injection impossible.

Task Conclusion :

In this Task we learned about SQL injection and parameterized queries and why/how they prevent SQL Injection.

Assignment Conclusion:

In this assignment we Learned about Web Application Reconnaissance and SQL injection to bypass non parameterized queries. This increased and polished my ability to use and learn more about cybersecurity.

• The End •

